COMPLETE DEVELOPER GUIDE

</>

HTML Mastery

From Basic to Professional — Every Concept Explained with Examples

4	40+	60+
LEVELS	CONCEPTS	EXAMPLES

HTML5 · Semantic Web · Accessibility · Performance

Table of Contents

A structured journey from beginner fundamentals to professional-level HTML

1. What is HTML?

HyperText Markup Language — the skeleton of every web page ever built

HTML stands for **HyperText Markup Language**. It is the standard language for creating web pages. HTML describes the *structure* of a web page using a system of elements represented by tags. Browsers parse HTML and render it into visual web pages.

A Brief History

Year	Version	Key Addition
1991	HTML 1.0	Basic tags: headings, paragraphs, links
1995	HTML 2.0	Forms, tables standardized
1997	HTML 3.2	Scripts, applets, enhanced tables
1999	HTML 4.01	CSS support, accessibility improvements
2014	HTML5	Semantic tags, audio/video, canvas, APIs
Present	HTML Living Standard	Maintained by WHATWG, continuously updated

How a Browser Renders HTML

When you open a webpage, the browser:

1. **Downloads** the HTML file from the server
2. **Parses** the HTML into a DOM (Document Object Model) tree
3. **Applies** CSS styles and runs JavaScript
4. **Paints** the final visual output on screen

> 💡 **Tip:** HTML is NOT a programming language — it doesn't have logic, loops, or variables. It's a *markup language* that defines the meaning and structure of content.

Your First HTML File

HTML

```html
<!-- Save this as index.html and open in any browser -->
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>I am learning HTML.</p>
```

```
  </body>
</html>
```

2. HTML Document Structure

Every valid HTML page follows a standard skeleton structure

The Full Boilerplate

HTML5 BOILERPLATE

```
<!DOCTYPE html>                  <!-- Declares HTML5 -->
<html lang="en">                   <!-- Root element, language -->
<head>
  <meta charset="UTF-8">           <!-- Character encoding -->
  <meta name="viewport"
        content="width=device-width,
        initial-scale=1.0">    <!-- Responsive design -->
  <meta name="description"
        content="Page description">
  <title>Page Title</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- All visible content goes here -->
  <script src="app.js"></script>  <!-- JS at end of body -->
</body>
</html>
```

Key Parts Explained

Element	Purpose	Required?
`<!DOCTYPE html>`	Tells browser this is HTML5	Yes
`<html lang="en">`	Root element; lang helps screen readers & SEO	Yes
`<head>`	Metadata container — not visible on page	Yes
`<meta charset="UTF-8">`	Supports all characters including emoji	Yes
`<meta name="viewport">`	Essential for mobile-responsive design	Yes
`<title>`	Shown in browser tab and search results	Yes
`<body>`	All visible page content goes here	Yes

📝 **Note:** Always put `<script>` tags just before the closing `</body>` tag. This ensures HTML renders before JavaScript executes, improving perceived load time.

3. Tags, Elements & Attributes

The building blocks of every HTML document

Anatomy of an HTML Element

HTML

```
<!-- Opening tag  Attribute name  Attribute value  Content  Closing tag -->
      <a           href=          "https://google.com"    >Visit Google</a>

<!-- Self-closing (void) elements have NO closing tag -->
<img src="photo.jpg" alt="A photo">
<br>
<hr>
<input type="text">
<meta charset="UTF-8">
```

Global Attributes — Work on ANY Element

Attribute	Description	Example
`id`	Unique identifier on the page	`id="header"`
`class`	CSS class name (reusable)	`class="btn primary"`
`style`	Inline CSS styles	`style="color:red"`
`title`	Tooltip on hover	`title="Click me"`
`lang`	Language of content	`lang="fr"`
`hidden`	Hides element	`hidden`
`tabindex`	Keyboard navigation order	`tabindex="1"`
`contenteditable`	Makes element editable	`contenteditable="true"`
`data-*`	Custom data attributes	`data-user-id="42"`

Block vs Inline Elements

Block Elements

Start on a new line, take full width

`<div>` Generic container

`<p>` Paragraph

Inline Elements

Flow within text, only take needed width

`<span>` Generic inline

`<a>` Link

<h1-h6>
Headings

Lists

Bold

Italic

<table>
Table

<form>
Form

<img>
Image

<button>
Button

4. Headings, Paragraphs & Text

Organizing and formatting text content on your pages

Heading Hierarchy

HTML

```
<h1>Main Page Title (use only ONE per page)</h1>
<h2>Section Heading</h2>
<h3>Sub-section</h3>
<h4>Deeper sub-section</h4>
<h5>Rarely needed</h5>
<h6>Smallest heading</h6>
```

⚠ **SEO Warning:** Use only ONE `<h1>` per page. It signals the main topic to search engines. Never skip heading levels (h1 → h3) — this breaks accessibility.

Text Formatting Tags

HTML

```
<p>Regular paragraph text.</p>

<strong>Bold — semantically important</strong>
<b>Bold — purely visual, no meaning</b>

<em>Italic — emphasis (screen readers stress it)</em>
<i>Italic — visual only (technical terms, titles)</i>

<mark>Highlighted text</mark>
<del>Strikethrough — deleted content</del>
<ins>Underline — inserted content</ins>
<small>Fine print / small text</small>
<sub>Subscript H<sub>2</sub>O</sub>
<sup>Superscript E = mc<sup>2</sup></sup>
<code>Inline code snippet</code>
<pre>Preserves    whitespace
    and line breaks</pre>
<blockquote cite="https://source.com">
  Famous quote here.
</blockquote>
<abbr title="HyperText Markup Language">HTML</abbr>
<br>  <!-- Line break -->
<hr>  <!-- Horizontal rule (thematic break) -->
```

Special Characters (HTML Entities)

Character	Entity	Use When
&	`&`	Literal ampersand in text
<	`<`	Less-than sign in content
>	`>`	Greater-than sign
	` `	Non-breaking space
©	`&copy;`	Copyright symbol
®	`&reg;`	Registered trademark
—	`&mdash;`	Em dash (—)

5. Links & 6. Images

Connecting pages and embedding visual content

Anchor Tag — All Use Cases

HTML

```
<!-- External link (new tab) -->
<a href="https://google.com" target="_blank"
   rel="noopener noreferrer">Google</a>

<!-- Internal page link -->
<a href="/about.html">About Us</a>

<!-- Jump to section (anchor) -->
<a href="#contact">Go to Contact</a>
<section id="contact">...</section>

<!-- Email link -->
<a href="mailto:hello@example.com">Email Me</a>

<!-- Phone link -->
<a href="tel:+1234567890">Call Us</a>

<!-- File download -->
<a href="report.pdf" download="my-report.pdf">Download PDF</a>
```

🔒 **Security:** Always add `rel="noopener noreferrer"` when using `target="_blank"`. Without it, the opened page can access and redirect your page via `window.opener`.

Images — Complete Reference

HTML

```
<!-- Basic image -->
<img src="photo.jpg" alt="A mountain landscape"
     width="800" height="600">

<!-- Responsive image with srcset -->
<img src="photo-800.jpg"
     srcset="photo-400.jpg 400w,
             photo-800.jpg 800w,
             photo-1600.jpg 1600w"
     sizes="(max-width: 600px) 400px,
            (max-width: 1200px) 800px,
            1600px"
     alt="Responsive photo"
     loading="lazy">
```

```
<!-- Picture element (art direction) -->
<picture>
  <source media="(max-width: 768px)"
          srcset="mobile.jpg">
  <source type="image/webp"
          srcset="photo.webp">
  <img src="fallback.jpg" alt="Description">
</picture>
```

7. Lists & 8. Tables

Organizing structured and tabular information

List Types

HTML

```html
<!-- Unordered List -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- Ordered List -->
<ol start="1" type="1">
  <li>First step</li>
  <li>Second step</li>
</ol>

<!-- Nested List -->
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
</ul>

<!-- Description List -->
<dl>
  <dt>HTML</dt>
  <dd>Markup language</dd>
  <dt>CSS</dt>
  <dd>Style sheets</dd>
</dl>
```

Complete Table

HTML

```html
<table>
  <caption>
    Student Grades
  </caption>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Score</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>95</td>
    </tr>
    <tr>
      <!-- Span columns -->
      <td colspan="2">
        Average: 95
      </td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Total</td>
      <td>95</td>
    </tr>
  </tfoot>
</table>
```

💡 **Best Practice:** Always use `<thead>`, `<tbody>`, `<tfoot>` in tables — not just for organization, but because browsers use them to scroll table bodies independently from fixed headers.

9. HTML Forms — Complete Reference

The primary way users interact with web applications

Form Structure & Input Types

HTML

```
<form action="/submit" method="POST"
      enctype="multipart/form-data">

  <!-- Text inputs -->
  <label for="name">Full Name</label>
  <input type="text" id="name" name="name"
         placeholder="John Doe" required
         minlength="2" maxlength="100">

  <input type="email"    name="email">
  <input type="password" name="pwd">
  <input type="number"   name="age" min="0" max="120">
  <input type="date"     name="dob">
  <input type="tel"      name="phone">
  <input type="url"      name="website">
  <input type="range"    min="0" max="100" step="5">
  <input type="color"    name="fav">
  <input type="file"     accept=".jpg,.pdf" multiple>
  <input type="hidden"   name="csrf" value="token123">

  <!-- Textarea -->
  <textarea name="message" rows="4" cols="50"
            placeholder="Your message..."></textarea>

  <!-- Select dropdown -->
  <select name="country">
    <optgroup label="Asia">
      <option value="in">India</option>
      <option value="jp">Japan</option>
    </optgroup>
  </select>

  <!-- Checkbox & Radio -->
  <input type="checkbox" name="agree" required>
  <input type="radio" name="gender" value="male">

  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
```

💡 **Accessibility:** Always pair `<label for="id">` with input `id="id"` . This allows clicking the label to focus the input — crucial for users with motor impairments and screen readers.

11. Semantic HTML5

Meaningful tags that improve SEO, accessibility, and code readability

Page Structure with Semantic Tags

HTML5 SEMANTIC STRUCTURE

```
<body>
  <header>                          <!-- Site header/logo/nav -->
    <nav aria-label="Main navigation">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about">About</a></li>
      </ul>
    </nav>
  </header>

  <main>                            <!-- Primary content (ONE per page) -->
    <article>                        <!-- Self-contained content -->
      <header>
        <h1>Article Title</h1>
        <time datetime="2024-01-15">Jan 15</time>
      </header>
      <section>                      <!-- Thematic grouping -->
        <h2>Introduction</h2>
        <p>Content here...</p>
      </section>
      <figure>                        <!-- Image with caption -->
        <img src="chart.png" alt="Sales chart">
        <figcaption>Q4 sales data</figcaption>
      </figure>
    </article>

    <aside>                          <!-- Sidebar / tangential content -->
      <p>Related articles...</p>
    </aside>
  </main>

  <footer>                          <!-- Site footer -->
    <address>                        <!-- Contact info -->
      <a href="mailto:hi@site.com">hi@site.com</a>
    </address>
  </footer>
</body>
```

Semantic Tag Quick Reference

`<header>`	`<nav>`	`<main>`
Page or section header	Navigation links	Primary content area

`<article>` Standalone content	`<section>` Thematic group	`<aside>` Sidebar content
`<footer>` Page or section footer	`<figure>` Media + caption	`<figcaption>` Caption for figure
`<time>` Date/time value	`<address>` Contact information	`<details>` Expandable content

13. Meta Tags, SEO & Social Sharing

Control how search engines and social platforms display your page

Essential Meta Tags

HTML — <HEAD> SECTION

```html
<head>
  <!-- Core -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Page Title | Brand Name</title>
  <meta name="description" content="150-160 char summary shown in Google">

  <!-- Robots control -->
  <meta name="robots" content="index, follow">
  <link rel="canonical" href="https://example.com/page">

  <!-- Open Graph (Facebook, LinkedIn, WhatsApp) -->
  <meta property="og:type"        content="website">
  <meta property="og:title"       content="Page Title">
  <meta property="og:description"  content="Description">
  <meta property="og:image"       content="https://example.com/og.jpg">
  <meta property="og:url"         content="https://example.com/page">

  <!-- Twitter Card -->
  <meta name="twitter:card"        content="summary_large_image">
  <meta name="twitter:title"       content="Page Title">
  <meta name="twitter:description"  content="Description">
  <meta name="twitter:image"       content="https://example.com/card.jpg">

  <!-- Favicon set -->
  <link rel="icon" type="image/svg+xml" href="/favicon.svg">
  <link rel="apple-touch-icon" href="/apple-icon.png">
  <link rel="manifest" href="/site.webmanifest">

  <!-- Theme color (browser UI on mobile) -->
  <meta name="theme-color" content="#1a1a2e">
</head>
```

🚀 **Pro Tip:** The Open Graph image should be **1200×630 pixels**. Twitter cards need at least **800×418px**. Use Twitter Card Validator and Facebook Debugger to test your tags.

18. Web Accessibility & ARIA

Building websites usable by everyone, including people with disabilities

ARIA Roles & Attributes

ARIA (Accessible Rich Internet Applications) attributes supplement HTML with extra semantic information for assistive technologies like screen readers.

HTML — ARIA EXAMPLES

```html
<!-- Landmark roles -->
<div role="banner">...</div>      <!-- same as <header> -->
<div role="navigation">...</div>  <!-- same as <nav> -->
<div role="main">...</div>        <!-- same as <main> -->

<!-- Labels for non-text elements -->
<button aria-label="Close dialog">x</button>
<nav aria-label="Breadcrumb">...</nav>

<!-- Describing via another element -->
<input aria-labelledby="hint1">
<span id="hint1">Enter your email</span>

<!-- States -->
<button aria-expanded="false"
        aria-controls="menu">Menu</button>
<ul id="menu" aria-hidden="true">...</ul>

<input aria-required="true"
       aria-invalid="false">

<!-- Live regions (dynamic updates) -->
<div aria-live="polite">
  Loading results...
</div>

<!-- Skip navigation (keyboard users) -->
<a href="#main"
   class="skip-link">Skip to main content</a>
```

Accessibility Checklist

Rule	Why It Matters
All images have `alt` text	Screen readers announce image descriptions
Color contrast ≥ 4.5:1	Readable for visually impaired users
All interactive elements are keyboard-focusable	Users who can't use a mouse

Forms have associated labels	Screen readers announce field purpose
Page has a meaningful `<title>`	First thing screen reader announces
Language declared on `<html>`	Correct pronunciation by text-to-speech
Focus indicator visible	Keyboard users need to see where they are

16–17. SVG & Canvas in HTML

Inline graphics — vector-based and pixel-based drawing

Inline SVG

SVG

```svg
<svg width="200" height="100"
     viewBox="0 0 200 100"
     xmlns="http://www.w3.org/2000/svg"
     role="img"
     aria-label="Logo">

  <!-- Rectangle -->
  <rect x="10" y="10"
        width="80" height="40"
        rx="5" fill="#e57373"/>

  <!-- Circle -->
  <circle cx="150" cy="50"
          r="30" fill="#1a1a2e"/>

  <!-- Path -->
  <path d="M10,80 Q100,20 190,80"
        stroke="#333"
        fill="none"
        stroke-width="2"/>

  <!-- Text -->
  <text x="100" y="55"
        text-anchor="middle"
        fill="white">Logo</text>
</svg>
```

Canvas API

HTML + JS

```html
<!-- HTML -->
<canvas id="myCanvas"
        width="400"
        height="200">
  Fallback text for no-canvas
</canvas>

<!-- JavaScript -->
<script>
const canvas =
  document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

// Rectangle
ctx.fillStyle = '#e57373';
ctx.fillRect(20, 20, 100, 60);

// Circle
ctx.beginPath();
ctx.arc(200, 100, 40, 0,
        Math.PI * 2);
ctx.fillStyle = '#1a1a2e';
ctx.fill();

// Text
ctx.font = '18px Arial';
ctx.fillStyle = '#333';
ctx.fillText('Hello Canvas',
             30, 130);
</script>
```

SVG vs Canvas — When to Use Which

	SVG	Canvas
Type	Vector (scalable)	Raster (pixel-based)
Performance	Great for <1000 elements	Great for 10,000+ elements
Interaction	Full DOM events per element	Manual hit-testing required

Best for	Icons, charts, diagrams, logos	Games, image editing, animations
Accessibility	Accessible with ARIA	Difficult, needs extra work

20. Web Components & 19. Performance

Reusable custom elements and loading optimizations

Web Components — Custom HTML Elements

HTML + JAVASCRIPT

```
<!-- 1. Define a template -->
<template id="user-card-template">
  <style>
    .card { background: #f9f9f9; padding: 16px; border-radius: 8px; }
  </style>
  <div class="card">
    <slot name="name">Default Name</slot>
    <slot name="role">Default Role</slot>
  </div>
</template>

<!-- 2. Define the custom element -->
<script>
class UserCard extends HTMLElement {
  connectedCallback() {
    const shadow = this.attachShadow({ mode: 'open' });
    const tmpl = document.getElementById('user-card-template');
    shadow.appendChild(tmpl.content.cloneNode(true));
  }
}
customElements.define('user-card', UserCard);
</script>

<!-- 3. Use it like native HTML! -->
<user-card>
  <span slot="name">Alice Smith</span>
  <span slot="role">Developer</span>
</user-card>
```

HTML Performance Optimization

HTML — PERFORMANCE PATTERNS

```
<!-- Lazy load images (native) -->
<img src="photo.jpg" loading="lazy">

<!-- Preload critical resources -->
<link rel="preload" href="font.woff2"
      as="font" type="font/woff2"
      crossorigin>

<!-- Prefetch next page -->
<link rel="prefetch" href="/next-page.html">
```

```
<link rel="dns-prefetch" href="//api.example.com">
<link rel="preconnect" href="https://fonts.googleapis.com">

<!-- Async & defer scripts -->
<script src="analytics.js" async></script>  <!-- runs ASAP -->
<script src="app.js"       defer></script>  <!-- runs after HTML parse -->

<!-- Image decode hint -->
<img src="hero.jpg" decoding="async"
     fetchpriority="high">
```

21. Structured Data & 22. Security

JSON-LD schema markup and protecting against HTML-based attacks

JSON-LD Structured Data (Schema.org)

Structured data helps search engines understand your content and show rich results (star ratings, recipes, FAQs, events in Google Search).

HTML — JSON-LD IN <HEAD>

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "HTML From Basic to Pro",
  "author": {
    "@type": "Person",
    "name": "Jane Doe"
  },
  "datePublished": "2024-01-15",
  "image": "https://example.com/og.jpg",
  "publisher": {
    "@type": "Organization",
    "name": "Dev Blog",
    "logo": {
      "@type": "ImageObject",
      "url": "https://example.com/logo.png"
    }
  }
}
</script>
```

HTML Security Best Practices

Attack	Prevention	HTML Technique
XSS (Cross-Site Scripting)	Never inject raw user data	Escape `<` `>` `&` in output
Clickjacking	Prevent embedding in iframes	CSP `frame-ancestors 'self'`
Open Redirect	Validate link destinations	Avoid user-controlled `href`
CSRF via forms	Use hidden token fields	`<input type="hidden" name="csrf">`
Malicious uploads	Restrict file types	`accept=".jpg,.png,.pdf"`

HTML — SECURITY HEADERS VIA <META>

```
<!-- Content Security Policy -->
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self';
               script-src 'self' https://cdnjs.com;
               style-src 'self' 'unsafe-inline'">

<!-- Prevent MIME type sniffing -->
<meta http-equiv="X-Content-Type-Options"
      content="nosniff">

<!-- Referrer policy -->
<meta name="referrer"
      content="strict-origin-when-cross-origin">
```

🚀 **Pro Tip:** Content Security Policy (CSP) headers are best set server-side (HTTP headers), not via meta tags. The meta tag version doesn't support some directives like `frame-ancestors`. Use meta CSP as a fallback only.

23. PWA Manifest & 14. Data Attributes

Installing web apps and embedding custom data in HTML

Progressive Web App — Web Manifest

A PWA can be installed on a user's home screen and run offline. The manifest file defines how it appears when installed.

SITE.WEBMANIFEST (JSON)

```
{
  "name": "My Awesome App",
  "short_name": "MyApp",
  "description": "A great web application",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#1a1a2e",
  "theme_color": "#e57373",
  "orientation": "portrait",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png",
      "purpose": "maskable" }
  ],
  "screenshots": [
    { "src": "/screenshot.png", "sizes": "1280x720",
      "type": "image/png", "form_factor": "wide" }
  ]
}
```

Data Attributes — Custom HTML Data

Store custom data directly in HTML elements using `data-*` attributes. Access them via JavaScript's `dataset` API.

HTML + JAVASCRIPT

```
<!-- HTML: Embed data in elements -->
<div id="product"
     data-product-id="SKU-1042"
     data-price="29.99"
     data-in-stock="true"
     data-category="electronics">
  Wireless Headphones
</div>

<button data-action="add-to-cart"
        data-sku="SKU-1042">Add to Cart</button>

<!-- JavaScript: Read with dataset -->
```

```
<script>
const el = document.getElementById('product');

// Access any data attribute via camelCase
console.log(el.dataset.productId);  // "SKU-1042"
console.log(el.dataset.price);      // "29.99"
console.log(el.dataset.inStock);    // "true"

// Set/update
el.dataset.price = "24.99";

// CSS can also read data attributes
// div[data-in-stock="false"] { opacity: 0.5; }

// Event delegation with data attributes
document.addEventListener('click', (e) => {
  if (e.target.dataset.action === 'add-to-cart') {
    addToCart(e.target.dataset.sku);
  }
});
</script>
```

12. Multimedia — Audio & Video

Native media elements without plugins

Audio Element

HTML

```
<audio
  controls
  preload="metadata"
  loop
  muted>
  <source
    src="song.mp3"
    type="audio/mpeg">
  <source
    src="song.ogg"
    type="audio/ogg">
  Your browser doesn't
  support audio.
</audio>
```

Video Element

HTML

```
<video
  width="640"
  height="360"
  controls
  poster="thumb.jpg"
  preload="none">
  <source
    src="video.mp4"
    type="video/mp4">
  <track
    kind="subtitles"
    src="sub.vtt"
    srclang="en"
    label="English">
</video>
```

iFrame — Embedding External Content

HTML

```
<!-- YouTube embed (best practice) -->
<iframe
  width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="Video title"
  loading="lazy"
  allow="accelerometer; autoplay;
         clipboard-write; encrypted-media;
         gyroscope; picture-in-picture"
  allowfullscreen
  sandbox="allow-scripts allow-same-origin
           allow-presentation">
</iframe>
```

💡 **Performance Tip:** Use the `loading="lazy"` attribute on iframes and images. This defers loading until the element is near the viewport, significantly improving initial page load time — especially for pages with many embeds.

HTML Details & Summary (Native Accordion)

HTML

```html
<details>
  <summary>Click to expand FAQ answer</summary>
  <p>The full answer is revealed here without any JavaScript!</p>
</details>

<!-- Open by default -->
<details open>
  <summary>Default open section</summary>
  <p>This content is visible immediately.</p>
</details>
```

HTML Pro Cheat Sheet

The most important rules for writing professional-grade HTML

The Golden Rules

✅ Always

- One `<h1>` per page
- `alt` on every image
- `lang` on `<html>`
- Labels paired with inputs
- `rel="noopener"` on new-tab links
- Semantic tags over `<div>` soup
- Validate HTML at validator.w3.org

❌ Never

- Skip heading levels (h1→h3)
- Use tables for layout
- Use inline styles for everything
- Forget the viewport meta tag
- Put scripts in `<head>` without defer
- Use `<b>` / `<i>` for semantic meaning
- Use empty `alt=""` for meaningful images

HTML5 Form Validation — No JavaScript Needed!

HTML — NATIVE VALIDATION

```
<input type="email"    required>                    <!-- valid email format -->
<input type="url"      required>                    <!-- valid URL -->
<input type="number"   min="1" max="100" step="1">  <!-- range check -->
<input type="text"     minlength="3" maxlength="50">  <!-- length check -->
<input type="text"     pattern="[A-Za-z]{3,}">      <!-- regex pattern -->
<input type="password" required minlength="8">     <!-- min 8 chars -->
```

Complete Tag Reference by Category

Category	Key Tags
Document	`<!DOCTYPE>` `<html>` `<head>` `<body>` `<title>` `<meta>` `<link>` `<script>` `<style>` `<base>`
Semantic Layout	`<header>` `<nav>` `<main>` `<article>` `<section>` `<aside>` `<footer>` `<figure>` `<address>`
Text Content	`<h1-h6>` `<p>` `<blockquote>` `<pre>` `<hr>` ` ` `<ul>` `<ol>` `<li>` `<dl>` `<dt>` `<dd>`
Inline Text	`<a>` `<strong>` `<em>` `<code>` `<mark>` `<small>` `<del>` `<ins>` `<sub>` `<sup>` `<abbr>` `<time>`
Media	`<img>` `<picture>` `<source>` `<audio>` `<video>` `<track>` `<iframe>` `<canvas>` `<svg>`
Forms	`<form>` `<input>` `<textarea>` `<select>` `<option>` `<optgroup>` `<label>` `<button>` `<fieldset>` `<legend>` `<datalist>`
Tables	`<table>` `<thead>` `<tbody>` `<tfoot>` `<tr>` `<th>` `<td>` `<caption>` `<col>` `<colgroup>`
Interactive	`<details>` `<summary>` `<dialog>` `<menu>` `<template>` `<slot>`

🎓 You've completed the HTML Basic to Pro Guide

Next steps: Learn CSS for styling → JavaScript for interactivity → React/Vue for building web apps